

PROJECT 3: SiteFlow Agent Platform

Full-Stack Frontend + Backend + AWS + LangGraph Multi-Agent System
Final Capstone | Autodesk / Bay Area Applied AI Interview Guide
Builds on Project 2 (Logistics RAG) — construction specs, materials, RFIs, logistics

1. Why This Project (Autodesk + Veteran + DS Fit)

Autodesk interviews for AI Engineer, ML Platform, and Construction Cloud roles test three things together: agentic workflows (multi-step tool use over AEC documents), cloud/backend engineering (AWS, REST APIs, file pipelines), and a real UI that talks to those APIs. Construction Cloud work is about RFIs, submittals, specifications, drawings-as-text, material lead times, and schedule risk — the same operational thinking as logistics. SiteFlow is a buildable replica of that product shape without requiring Autodesk Forge credentials.

You already have Project 2 (document RAG + tools). This project adds: a separate FastAPI backend, a React frontend, AWS for files and optional LLM hosting, LangGraph multi-agent orchestration, human-in-the-loop approval, and an integration contract you can defend in system-design interviews.

What interviewers should see in the repo

- Frontend: React + Vite chat + project dashboard + file upload + citation panel.
- Backend: FastAPI REST + streaming chat, auth stub, agent graph, tool layer.
- AWS: S3 document store, IAM least-privilege, optional Bedrock LLM, DynamoDB session state, CloudWatch logs.
- Agents: LangGraph supervisor + Spec/RAG agent + Material/Logistics agent + Risk agent + HITL node.
- Data science: chunking/eval set, faithfulness notes, embedding choice, latency/cost trade-offs.

2. Product Definition

SiteFlow is a project-controls assistant for a single construction job. A superintendent or project engineer uploads specs, RFI PDFs, and a materials CSV. They ask: “What is the lead time language for structural steel?” or “If rebar demand is 12,000 units and holding cost is \$4, what EOQ should we use?” or “Which open RFIs block the foundation pour?” The supervisor agent routes to the right specialist, tools may run, and the UI shows answer + sources + which agent ran + an Approve / Reject button for high-impact recommendations.

Personas (use these words in interviews)

- Project engineer / document controller — search specs and RFIs.
- Superintendent — material availability and field constraints.
- Estimator / logistics planner — EOQ, safety stock, lead-time risk (your Project 2 strength).

3. Architecture (Say This on a Whiteboard)

```
[React UI] --HTTPS JSON/SSE--> [FastAPI on local or ECS]
FastAPI --> LangGraph (shared AgentState)
Supervisor --> spec_rag | materials | risk | human_review | respond
spec_rag --> Chroma/FAISS + embeddings; docs live in S3, indexed locally or via Lambda
materials --> tools: eoq, safety_stock, sku_lookup (CSV in S3)
risk --> simple rules + retrieved delay language from specs
State checkpoint --> local sqlite first; DynamoDB if you add AWS persistence
LLM --> Ollama locally; swap to Bedrock Claude/Titan when AWS keys exist
```

Integration contract: frontend never talks to LangGraph or S3 directly. All file upload, chat, and approval go through FastAPI. That is the Autodesk-style platform split (UI product vs AI platform).

4. Stack (Buildable on the Mac You Configured)

Layer	Choice	Interview talking point
-------	--------	-------------------------

Frontend	React 18 + Vite + Tailwind	Autodesk loops often include React + API integration
Backend	FastAPI + Pydantic v2 + uvicorn	Typed API contracts, OpenAPI, streaming
Agents	LangGraph + LangChain tools	Stateful graph, HITL, routing — Autodesk agent questions
LLM local	Ollama llama3.1:8b	Dev without spend; same interface as Bedrock wrapper
LLM AWS	Bedrock Claude Haiku / Titan Embed	Show you can swap providers behind one interface
Files	S3 + local fallback ./data	Presigned upload URLs, event-driven ingest
Vectors	Chroma persist dir (then OpenSearch later)	Know the upgrade path to OpenSearch Serverless
Sessions	SQLite checkpointer, optional DynamoDB	Durable agent state across requests
Observability	Structured logs + optional LangSmith	Trace agent steps; CloudWatch in AWS mode

5. Repository Layout (Create This First)

```

siteflow/
├── backend/
│   ├── app/main.py # FastAPI app, CORS, routers
│   ├── app/api/docs.py # POST /projects/{id}/documents (upload)
│   ├── app/api/chat.py # POST /chat GET /chat/{id}/stream
│   ├── app/api/approvals.py # POST /approvals/{id}
│   ├── app/agents/state.py # TypedDict AgentState
│   ├── app/agents/graph.py # LangGraph compile
│   ├── app/agents/supervisor.py
│   ├── app/agents/spec_agent.py
│   ├── app/agents/materials_agent.py
│   ├── app/agents/risk_agent.py
│   ├── app/tools/inventory.py # reuse Project 2 EOQ / safety stock
│   ├── app/rag/ingest.py
│   ├── app/rag/retriever.py
│   ├── app/aws/s3.py # upload, presign, download
│   ├── app/aws/bedrock.py # optional LLM + embeddings
│   ├── app/aws/dynamo.py # optional checkpoints
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/App.jsx
│   ├── src/api/client.js # fetch wrappers
│   ├── src/components/Chat.jsx
│   ├── src/components/Sources.jsx
│   ├── src/components/Upload.jsx
│   ├── src/components/ApprovalCard.jsx
│   ├── package.json
│   └── infra/ # optional: IAM policy JSON + S3 bucket notes
├── data/sample/ # 6-10 PDFs + materials.csv
├── eval/questions.json
└── README.md

```

6. Environment Setup

6.1 Backend

```

cd backend && python -m venv .venv && source .venv/bin/activate
pip install fastapi uvicorn pydantic python-multipart langchain langchain-ollama \
langgraph langchain-community chromadb pypdf boto3 python-dotenv sse-starlette

```

6.2 Frontend

```

cd frontend && npm create vite@latest . -- --template react
npm install
# add a simple fetch client; Tailwind optional

```

6.3 AWS (can start in local mode)

Create a free-tier AWS account. Make an IAM user with only: s3:PutObject/GetObject/ListBucket on one bucket siteflow-docs-<yourname>, optional bedrock:InvokeModel, optional dynamodb on one table. Never commit keys. .env: AWS_REGION=us-west-2, S3_BUCKET=..., USE_AWS=false for week 1.

```
SITEFLOW_MODE=local|aws
OLLAMA_MODEL=llama3.1:8b
AWS_REGION=us-west-2
S3_BUCKET=siteflow-docs-YOURNAME
BEDROCK_MODEL_ID=anthropic.claude-3-haiku-20240307-v1:0
```

7. Build Order (Do Not Skip Local Mode)

Day 1–2 — Backend skeleton + Project 2 reuse

Stand up FastAPI health check. Port ingest.py and tools.py from Project 2 into backend/app/rag and backend/app/tools. POST /projects/demo/documents accepts multipart PDF/CSV and writes to .data. GET /projects/demo/documents lists files. Write 3 pytest tests: upload stores file, EOQ formula, retriever returns k chunks.

Day 3–4 — LangGraph graph

AgentState fields: messages, route, sources, tool_result, needs_approval, recommendation, project_id. Supervisor uses a short classifier prompt or keyword+LLM hybrid: specs/RFI → spec_rag; EOQ/stock/sku → materials; delay/risk/weather/pour → risk; else respond. Each specialist node returns updated state. Compile graph with MemorySaver. Expose POST /chat {project_id, message, thread_id}. Return answer, sources[], agents_used[], needs_approval.

Day 5 — Frontend integration

Vite React app on :5173, proxy /api to FastAPI :8000. Pages: Project home (file list + upload), Chat (message list, source chips), Approval card when needs_approval is true. client.js functions: uploadDocument, listDocuments, sendChat, submitApproval. Show loading and error states. This is the integration you will demo.

Day 6 — AWS integration (minimum viable cloud)

- 1) Create S3 bucket, block public access, enable default encryption.
- 2) app/aws/s3.py: upload_fileobj, generate_presigned_url (PUT 5 min), list_prefix.
- 3) When USE_AWS=true, uploads go to s3://bucket/projects/{id}/docs/.
- 4) Optional: Bedrock wrapper implementing the same .invoke(messages) interface as ChatOllama.
- 5) Log request_id, project_id, route, latency_ms to stdout (CloudWatch later).
- 6) Document IAM policy JSON in infra/iam-siteflow.json.

Day 7 — Eval, README, demo video

12 gold questions in eval/questions.json (4 spec, 4 materials, 2 risk, 2 out-of-scope). Score routing accuracy and groundedness by hand. README: architecture diagram, local vs AWS mode, example Q&A, limitations, cost notes. 2–3 min Loom: upload spec → ask RFI question → ask EOQ → show approval card.

8. Sample Data To Invent (Keep It Legal and Small)

Create 6–8 short PDFs yourself (2–6 pages). Do not scrape copyrighted Autodesk manuals. Suggested titles:

- Division 03 Cast-in-Place Concrete — spec excerpts (curing, strength, weather limits).
- Division 05 Structural Steel — lead times, mill certs, bolt spec.
- RFI-014 Foundation Pour Hold — open RFI blocking slab on grade.
- Submittal log excerpt — rebar, anchor bolts, vapor barrier.
- Site logistics plan — laydown, deliveries, gate hours.
- materials.csv — sku, description, on_hand, lead_days, unit_cost, annual_demand.

9. API Contract (Frontend Must Use Only These)

```
POST /api/projects/{id}/documents multipart file
GET /api/projects/{id}/documents
POST /api/chat {project_id, thread_id, message}
```

```
→ {answer, sources:[{title,page,snippet}], agents_used[], needs_approval, recommendation}  
POST /api/approvals/{thread_id} {approved: bool, note}  
GET /api/health
```

CORS: allow localhost:5173. Validate file types (pdf, csv, txt) and 10 MB max. Never return raw stack traces.

10. Agent and Tool Specs

Supervisor — one-token or short JSON route: spec | materials | risk | chat. If recommendation changes spend or schedule, set needs_approval=true.

Spec agent — retrieve k=4 chunks, cite source titles, refuse if not in docs.

Materials agent — tools: calculate_eoq(D,S,H), safety_stock(std,lt,z), lookup_sku(sku). Reuse Project 2 formulas. After tool result, write a one-paragraph field recommendation.

Risk agent — retrieve weather/hold/RFI language; output risk_level + blocked_activities list.

HITL node — if needs_approval and not yet decided, pause and return card to UI. On POST /approvals, resume graph with human decision in state.

11. AWS Pieces You Must Be Able to Explain

- S3: bucket policy vs IAM user policy; Block Public Access; SSE-S3; prefix layout projects/{id}/docs/.
- Presigned URL: frontend can PUT directly to S3; backend still creates the URL so keys stay server-side.
- Bedrock vs Ollama: same chat interface; Bedrock adds IAM auth, region, model IDs, throughput limits.
- DynamoDB: partition key thread_id, stores serialized graph checkpoint; TTL optional.
- Lambda (stretch): S3 ObjectCreated → ingest chunks. You can describe this even if you only run ingest in FastAPI.
- ECS Fargate + ALB (stretch talk track): containerize FastAPI; React on CloudFront/S3 or Amplify.
- Cost: embeddings + chat tokens; cache embeddings; Haiku for routing, larger model only for final answer.

12. Autodesk Interview Map (How They Hire in 2026)

Typical AI / platform / Construction Cloud loop: recruiter screen → hiring manager (projects) → live coding (Python and sometimes React) → ML / agent system design → domain + behavioral. Topics that show up: multi-agent breakdown of user instructions, RAG over specs/drawings, function calling against enterprise data, AWS beyond VMs, React talking to APIs, evaluation and hallucination control, HITL for high-stakes actions.

12.1 Map SiteFlow to Autodesk language

- Spec/RFI RAG = Construction Cloud document intelligence / Assistant answer-finding.
- Materials + EOQ tools = agentic actions on project data (not just chat).
- Supervisor graph = Autodesk-style orchestrator routing to the right workflow.
- Approval card = responsible AI / human-in-the-loop before changing the job.
- FastAPI + React = platform vs product split used on AI/ML Platform teams.
- S3 + Bedrock = how they expect cloud-native AI services to look.

12.2 Live coding drill (30–45 min)

Python: parse a materials.csv, compute EOQ, return JSON. Debounce a React search box. Implement GET list + POST upload handler. SQL-ish: RFIs by status and spec section. Write a Pydantic model for ChatRequest and reject empty message.

12.3 System design prompt they will ask (practice this exact one)

“Design an assistant that answers questions across 10,000 construction PDFs and can create an RFI draft. Cover ingest, retrieval, agents, permissions, evaluation, and cost.” Answer with SiteFlow scaled up: S3 + event ingest Lambda, OpenSearch kNN, Bedrock, LangGraph supervisor, project-scoped IAM, audit log, eval harness, HITL before any write-back to the system of record.

13. ChatGPT Mock Interview Pack (Autodesk Persona)

Master system prompt — paste every session

You are a senior AI Engineer / hiring manager on Autodesk Construction Cloud or Autodesk AI Platform in the Bay Area. You interview a veteran with a Master's in data science, logistics operations experience, a completed logistics RAG project, and a new full-stack SiteFlow multi-agent app (React + FastAPI + LangGraph + AWS S3/Bedrock). Probe architecture, AWS choices, agent routing failures, RAG evaluation, frontend-backend contract, and construction domain. After each answer score 1–5 on technical depth, production judgment, and communication. Judge content, not speech fluency. Start by asking me to walk through SiteFlow end to end.

Project deep-dive (ask these in order)

1. Walk through one user question from the React UI to S3, the graph, tools, and back. Name every hop.
2. Why is the frontend forbidden from calling S3 or LangGraph directly?
3. How does the supervisor decide spec vs materials vs risk? What is the failure mode if it mis-routes?
4. How did you reuse Project 2 tools inside a new backend without copying a blob of dead code?
5. Show the AgentState schema. What belongs in state vs a database vs S3?
6. If two users upload different projects, how do you isolate vectors and files?
7. When would you move Chroma to OpenSearch Serverless? What breaks if you do it too early?
8. How do you keep answers grounded in specs instead of the model's general construction knowledge?
9. Explain presigned uploads vs proxying the file through FastAPI. Which did you pick and why?

10. Where does human approval sit in the graph? What happens if the user never clicks Approve?
11. How would you add a write action like “create RFI” without letting the agent mutate data unsupervised?
12. What did you measure in eval/questions.json? Routing accuracy vs faithfulness vs latency?

AWS + platform

1. Write the IAM actions your backend user actually needs. What did you explicitly deny?
2. Bedrock invoke failed in us-east-1 but your bucket is us-west-2. How do you design regions?
3. How do you rotate keys and keep .env out of git?
4. Sketch ECS + ALB + CloudFront for this app. What is stateless vs stateful?
5. A 200 MB drawing package lands in S3. Why is FastAPI ingest on the request path a bad idea?

Data science / ML

1. Why recursive character split over semantic split for specification PDFs?
2. How would you detect that embeddings drifted after a model upgrade?
3. Give a precision/recall style definition for “correct citation” on this product.
4. When is a small local 8B model acceptable vs Bedrock for routing vs final drafting?

Behavioral (veteran + Autodesk values)

Tell me about a time incomplete information still required a field decision. How would SiteFlow change that?

Describe a disagreement with a stakeholder about shipping a risky automation. Where did you put the human gate?

Why Autodesk / Construction Cloud instead of a generic chatbot company?

Full 25-minute role-play prompt

Run a realistic Autodesk AI Engineer interview. Start with “Walk me through SiteFlow.” Then: one live-style question (design the POST /chat payload and error cases), one system-design scale-up to 10k PDFs, one AWS IAM/S3 question, one RAG evaluation question, one behavioral about logistics/veteran decision-making. After each answer, score and ask a follow-up. End with hire / lean-hire / no-hire and the top two gaps. Begin now.

14. Definition of Done

- React UI: upload, chat, sources, approval card
- FastAPI: documents + chat + approvals + health
- LangGraph: supervisor + 3 specialists + HITL flag
- Local mode works with zero AWS
- AWS mode: at least S3 upload/list working with IAM notes
- Bedrock optional but interface exists (or documented stub)
- 12 eval questions scored
- README + 3-minute demo
- Can answer Section 13 questions out loud

15. What Not To Do

Do not start with EKS/Kubernetes. Do not require Autodesk Forge APIs. Do not build five microservices. Do not skip the React UI. Do not hardcode AWS keys. Do not claim the agent “creates RFIs in ACC” unless you only simulated the draft.

Ship a thin vertical slice first: one PDF, one CSV, one chat path, one tool, one approval. Then add AWS S3. That slice is enough to start Autodesk-style interviews while you harden the rest.

Project 3 of the job-ready sprint | SiteFlow Agent Platform | React + FastAPI + LangGraph + AWS